TECHNICAL MANUALS

Controlli Numerici Industriali

CNi PCQUO MANUAL

Author: Davide Pelloni

Date: 10/04/2001

MANUAL REVISION **1.0.0.0**

MANUAL ISSUE **10th april 2001**

ARCHIVE NUMBER **X2763**

PUBLICATION ISSUED BY:

C.N.I. Controlli Numerici Industriali S.r.l.

Registered Offices
Via Carpanelli, 24 - 40011 ANZOLA dell'EMILIA (Bo)
Tel. 051.73.48.90 - Telefax 051.73.48.91

Main Offices
Via dell'Artigianato, 1 - 48011 ALFONSINE (Ra)
Tel. 0544.81.693 - Telefax 0544.81.337

Tax Number
02480600374
R.S.T. Bologna n. 41287 -
CCIAA R.D. n. 294814 - M 138519

CONTENTS

1 Preliminary Information

This document contains the instructions for installation and use of the software library **CNi PCQUO** for Microsoft® Window 98/NT/2000 ®.
In this document it is assumed that the reader is familiar with computers, and also has sufficient knowledge of the operating and software development environment. For information on the installation, start-up and use of Windows ® applications, please consult the on-line guide and documentation provided with said applications.

1.1 Description of the software package CNi PCQUO

The **CNi PCQUO** software package consists of a library of software functions using which it is possible to develop applications, resident on the PC, that interface with the **Positions** and **List** application software resident on the XNC type numerical controls manufactured by C.N.i. srl. Connection between the Numerical Control and the Personal Computer uses a serial line or network card, and TCP/IP communication protocol.

AVVERTENZA: The CNi PCQUO package allowes interface with XNC's with Machine Version 2.0.0.0 or higher

1.2 System requirements

The **CNi PCQUO** software package can be installed on any Personal Computer with at least the following characteristics:

- Floppy Disk drive or CD-ROM
- Windows 98/NT/2000 ® operating system
- CPU 486 (PENTIUM recommended)
- 16 Mbyte RAM (32 Mbyte recommended)
- Hard disk with at least 1 Mbyte free space available

AVVERTENZA: the system requirements given above are the minimum requirements, without which correct installation of the software package cannot be guaranteed

2 Library Functions

The library **Cni PCQUO** contains all the software functions allowing data exchange between the software applications resident in a PC and the **Positions** and **List** application programs on the XNC numerical control.
For a detailed description of the **Positions** and **List** application programs, please see the user's manual for the XNC numerical control manufactured by C.N.i. srl.

2.1 Description of the library functions

Interaction between a software resident on the PC and the Positions and List application programs resident on the NC requires a "conversation protocol" based on the transmission and reception of "commands".
The term "commands" indicates both requests for information, for example the current machine status, and the enabling of functions that influence running of the machining programs, for example selection of the program to be run.

The conversation protocol can be summed up as follows:

- The resident PC software sends a command to the NC using a suitable CNi PCQUO library function
- The NC responds to the command received, loading suitable preset variables and providing a data item which is returned by the calling function.

The value returned by the library functions can be one of the following:

- **-**1*: Command transmission error:* the value -1 indicates that the command has not been transmitted correctly, for example due to problems in the connection between PC and NC
- ***-2:*** *Error receiving response to the command sent:* the value -2 indicates that the command has not been received correctly, for example due to problems in the connection between PC and NC
- ***>0:*** *Command management error:* a value higher than 0 indicates that the command is considered to be incorrect, and is not managed by the NC. This type of error occurs typically when the data supplied with the command are incorrect or incomplete
- ***0:*** *Command OK*: a value of 0 indicates that the command has been managed correctly

Certain library functions foresee input of the name of a program file or list file. The name of the program or list files can contain letter and numbers, but must not contain the following characters:

- " * " asterisk
- " " space
- control characters (i.e. tab, line feed, etc.)

It is also impossible to use the character strings "NULL", "NONE", "None" and "none" as program names.

❖ Initialisation of XNC Communication channel

int openQuoChannel (char * nomemacc)

PARAMETRI:

- ***nomemacc****:* string containing the name of the XNC machine with which connection is to be made (hostname).

VALORI DI RITORNO:

- 0 if communication with the XNC is enabled correctly
- <> 0 if connection fails

NOTE:

Call-up of this function initialises the XNC communication channel, enabling interaction with the resident application software Positions and List.
If call-up of this function cannot enable connection between the PC and the XNC the function will indicate ERROR. The string containing the name of the numerical control to which the PC must be connected has to end with the end of string character NULL.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

val_ret = openQuoChannel("_xnc");
if ( val_ret != 0 )
    printf( "XNC Connection Error =%d" , val_ret );
else
    printf( "XNC Connection Open" );
}
```

❖ Close XNC Communication channel

void closeQuoChannel(void)

PARAMETRI:
None.

VALORI DI RITORNO:
None.

NOTE:
Call-up of this function closes the XNC communication channel opened using the function ***openQuoChannel***.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int    val_ret;

val_ret = openQuoChannel("_xnc");
if ( val_ret == 0 )
        {
        closeQuoChannel();
        printf( "XNC Connection Closed" );
        }
}
```

❖ Message to be displayed on the XNC.

int sendQuoMessage(char * sigla,char * mess)

PARAMETRI:

- ***sigla***: string containing the MESSAGE CODE
- ***mess***: string containing the MESSAGE STRING

VALORI DI RITORNO:

- 0 if the message has been displayed by the XNC
- <> 0 in case of error

NOTE:

Call-up of this function sends two strings to the **Positions** application program, which displays them in the **Messages** window. The first string, called MESSAGE CODE, has to indicate the name of the machine connected to the XNC, the second, called MESSAGE STRING, forms the actual message to be displayed.
The maximum number of characters in the MESSAGE CODE is defined in the define *MAX_SIGLA_MSG_SQUO* in file *pcquote.h*; The maximum number of characters in the MESSAGE STRING is defined in the define MAX_STRINGA_MSG_SQUO in file *pcquote.h.*
The strings given as input parameters must terminate with the end of string character NULL.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoMessage("PC1" , "PC CONNECTED" );
      if ( val_ret != 0 )
            printf( "Message transimssion Error=%d" , val_ret );
      else
            printf( "Correctly sent Message" );
      }
}
```

❖ **Request for XNC Status information.**

int sendQuoGetFlagsStato(int *fdstn,int *fmaut,int *fstart)

PARAMETRI:

- ***fdstn***: address of the list present data memory storage variable
- ***fmaut***: address of the automatic machine mode data memory storage variable.
- ***fstart***: address of the machine in start data memory storage variable.

VALORI DI RITORNO:

- 0 if the data is available
- <> 0 in case of error

NOTE:

After call-up of this function, the following values are written to the input memory addresses:

- ***fdstn***: 1 if the list is present, otherwise 0
- ***fmaut***: 1 if the active machine mode is AUTOMATIC, otherwise 0
- ***fstart***: 1 if the machine is running, otherwise 0

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret,flag_dstn,flag_autom,flag_start;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoGetFlagsStato(&flag_dstn, &flag_autom, &flag_start );
      if ( val_ret != 0 )
            printf( "XNC Status - Information Request Error=%d" , val_ret );
      else
            {
            printf( "Worklist Available=%d" , flag_dstn);
            printf( "Automatic Mode=%d" , flag_autom);
            printf( "Start Working Cycle=%d", flag_start );
            }
      }
}
```

❖ Start in single program mode.

int sendQuoStartProg(void)

PARAMETRI:

None.

VALORI DI RITORNO:

- 0 if the command has been sent successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function sends the START SINGLE PROGRAM command to the XNC. SINGLE PROGRAM mode is available when the List is not enabled. If machine conditions allow, the XNC enables running of the selected program in this mode.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoStartProg();
      if ( val_ret != 0 )
           printf( "Single Program Start Error=%d" , val_ret );
      else
           printf( "Single Program Start Performed" );
      }
}
```

❖ **Select single program.**

int sendQuoSelProg(char *nomep,char *parap)

PARAMETRI:

- ***nomep***: string containing the name of the program to be selected
- ***parap***: string containing the program comments

VALORI DI RITORNO:

- 0 if the command has been sent successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function selects a program in SINGLE PROGRAM mode on the XNC. The input parameters must be two strings; the first, ***nomep***, must contain the name of the program to be selected, the second, ***parap***, the comments for that program. SINGLE PROGRAM mode is available when the List is not enabled. If ***parap*** is set to NULL, the comment for the program will not be selected. If the XNC allows this, the program comment string can include assignment of certain parameters to be used by the program.
The strings given as input parameters must terminate with the end of string character NULL.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
char  *nprog="PROGR1";
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoSelProg(  nprog , NULL );
      if ( val_ret != 0 )
           printf( "Selection Error on Single Program=%d" , val_ret );
      else
           printf( "Single Program Selection Performed=%s" , nprog );
      }
}
```

❖ Request single program.

int sendQuoGetProg(char *prgsel)

PARAMETRI:

- ***prgsel***: address of the memory storage variable for the string containing the name of the single program selected

VALORI DI RITORNO:

- 0 if the request has been satisfied
- <> 0 in case of ERROR

NOTE:

Call-up of this function copies a string containing the name of the selected program to the XNC in SINGLE PROGRAM mode, starting from the memory address specified as an input parameter. This information is available when the list is not enabled. The maximum number of characters in the string copied is equal to the value in the define *MAX_STRINGA_PROG_SQUO* in file *pcquote.h*.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
char progr[MAX_STRINGA_PROG_SQUO];
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoGetProg( &prog[0] );
      if ( val_ret != 0 )
           printf( "Single Program Request Error=%d" , val_ret );
      else
           printf( "Single Program Name = %s", progr );
      }
}
```

❖ **Start list.**

int sendQuoStartDstn(void)

PARAMETRI:
None.

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:
Call-up of this function sends the START command to the list, ordering running of the programs it contains. If the list is not enabled the function will return an error.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoStartDstn();
      if ( val_ret != 0 )
            printf( "Worklist Start Error=%d" , val_ret );
      else
            printf( "Executing Worklist" );
      }
}
```

❖ Send list line.

int sendQuoRowDstn(char *strriga)

PARAMETRI:

- ***strriga***: string containing the line to be inserted in the list; it must end with NULL.

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function inserts a program line at the end of the list. The line must contain the program name, the amount, the piece counter and the comment; these fields must be located, within the function input string, according to the bar code configuration (see XNC manual).

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
char  *row_str="    PROGR      1    0    $ PAR=100 $"
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoRowDstn( row_str );
      if ( val_ret != 0 )
           printf( "Worklist Line Send Error=%d" , val_ret );
      else
           printf( "Worklist Line Sent=%s", row_str );
      }
}
```

❖ **Send program to list.**

int sendQuoProgDstn(char *nomep,char *parap)

PARAMETRI:

- ***nomep***: string containing the name of the program to be added
- ***parap***: string containing the comment for the program to be added

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function adds a program line at the end of the list. The line contains the name of the program to be run and the comment for the program itself. The amount and the piece counter are set automatically by the XNC to 1 and 0, respectively. If the parameter ***parap*** is set to NULL the comment field for the line will be left empty.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
char  *nprog="PROGR1";
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoProgDstn(  nprog , NULL );
      if ( val_ret != 0 )
           printf( "Worklist Program Send Error=%d" , val_ret );
      else
           printf( "Worklist Program Sent=%s" , nprog );
      }
}
```

❖ Request program name for list line.

int sendQuoGetProgDstn(char *prgrig)

PARAMETRI:

- ***prgrig***: address of the memory storage variable for the string containing the name of the program in the edit line

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function adds the name of the program found in the edit line of the list, to the parameter ***prgrig***. This information is available when the list is enabled. The maximum number of characters in the storage string must be at least equal to the value of the define *MAX_STRINGA_PROG_SQUO* in file *pcquote.h*.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
char  progr[MAX_STRINGA_PROG_SQUO];
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoGetProgDstn( &progr[0] );
      if ( val_ret != 0 )
           printf( "Edit Line: Request Program Error=%d" , val_ret );
      else
           printf( "Edit Line: Program Name= %s", progr );
      }
}
```

❖ Delete list line.

int sendQuoDelRowDstn(int riga)

PARAMETRI:

- ***riga***: number of the line to be deleted; assign the value -1 to delete the whole list.

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function deletes the list line indicated in the input parameter ***riga***. If the input parameter ***riga*** is given a value of -1 the whole list is deleted. It is not possible to delete when the XNC is running.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoDelRowDstn(  -1 );
      if ( val_ret != 0 )
           printf( "Worklist Deleting Error=%d" , val_ret );
      else
           printf( "Worklist Deletion performed" );
      }
}
```

❖ Select edit line in list.

int sendQuoSetPosDstn(int riga)

PARAMETRI:

- ***riga***: line number to be selected in the list

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function selects the line in the list indicated by the input parameter. The selected line can then be modified. This command is only available when the list is enabled.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;
int     riga=0;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoSetPosDstn(riga);
      if ( val_ret != 0 )
           printf( "Edit Line: Selection Error=%d" , val_ret );
      else
           printf( "Worklist Edit Line Selected=%d", riga );
      }
}
```

❖ **Request edit line in list.**

int sendQuoGetPosDstn(int *rigc)

PARAMETRI:

- ***rigc***: address of the storage memory variable for the line number being modified in the list.

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function types the number for the line being edited in the list into the input parameter ***rigc***. This information is available when the list is enabled.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;
int     rigc;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoGetPosDstn( &rigc );
      if ( val_ret != 0 )
            printf( "Edit Line: Request Error=%d" , val_ret );
      else
            printf( "Worklist Edit Line=%d", rigc );
      }
}
```

❖ Read list file.

int sendQuoLoadDstn(char *nomef)

PARAMETRI:

- ***nomef***: name of the list file to be loaded

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function loads the list file indicated in the input parameter ***nomef***. This file is considered by the XNC as a file that is being edited.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoLoadDstn(  "DISTINTA" );
      if ( val_ret != 0 )
           printf( "Worklist File Read Error=%d" , val_ret );
      else
           printf( "Worklist File Reading Performed" );
      }
}
```

❖ **Write list file.**

int sendQuoSaveDstn(char *nomef)

PARAMETRI:

- ***nomef***: name of the list file to be stored

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function causes storage of the list file indicated in the input parameter ***nomef***. If the input parameter ***nomef*** is set to NULL the write will be carried out on the file that is currently being edited.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoSaveDstn(  "NULL" );
      if ( val_ret != 0 )
            printf( "Worklist File Write Error=%d" , val_ret );
      else
            printf( "Worklist File Writing Performed" );
      }
}
```

❖ Request information on the list.

int sendQuoGetInfDstn(char *nomed,int *rigd)

PARAMETRI:

- ***nomed***: address of the storage memory variable for the name of the list file being edited
- ***rigd***: address of the storage memory variable for the total number of lines in the list

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function writes the name of the list file being edited and the total number of lines in the list, respectively, in the input parameters ***nomed*** and ***rigd***. This information is only available when the list is enabled. The size of the string containing the list file name must be at least MAX_STRINGA_DSTN_SQUO characters. This limit is defined in the file *pcquote.h*

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{

char  ndstn[MAX_STRINGA_DSTN_SQUO];
int     val_ret,rigd;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoGetInfDstn( &ndstn[0], &rigd );
      if ( val_ret != 0 )
           printf( "Worklist Information Request Error=%d" , val_ret );
      else
           printf( "Worklist Filename in Edit= %s ; Worklist Lines Number=%d", ndstn, rigd );
      }
}
```

❖ Select amount and piece counter.

int sendQuoSetQntaCont(int riga,int qnta,int cont)

PARAMETRI:

- ***riga***: line number in the list
- ***qnta***: value for the amount to be selected
- ***cont***: value for the piece counter to be selected

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function is used to assign the amount and/or the piece counter for the list line indicated in the input parameter ***riga***. If the parameter ***qnta*** is set to -1 no value is given for the amount on that line in the list. If the parameter ***cont*** is given no value, no value is assigned to the piece counter on that line in the list.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoSetQntaCont( 0,1,0 );
      if ( val_ret != 0 )
            printf( "Preset and Counter Selection Error=%d" , val_ret );
      else
            printf( "Preset and Counter Selection Performed" );
      }
}
```

❖ Request amount and piece counter.

int sendQuoGetQntaCont(int riga,int *dqta,int *dcnt)

PARAMETRI:

- ***riga***: number of the line in the list for which the amount and the piece counter is to be obtained.
- ***dqta***: address for the memory variable in which the value of the quantity field for the list line specified in input parameter ***riga*** is stored
- ***dcnt***: address for the memory variable in which the value of the piece counter field for the list line specified in input parameter ***riga*** is stored

VALORI DI RITORNO:

- 0 if the command has been performed successfully
- <> 0 in case of ERROR

NOTE:

Call-up of this function writes the value of the quantity and piece counter fields for the list line specified in input parameter ***riga*** into the input parameters ***dqta*** and ***dcnt***, respectively. This information is only available when the list is enabled.

ESEMPIO:

```
#include <string.h>
#include <stdio.h>
#include <pcquote.h>

void main(void)
{
int     val_ret;
int     qnta,cont;

if( openQuoChannel("_xnc") == 0 )
      {
      val_ret = sendQuoGetQntaCont( 0, &qnta, &cont );
      if ( val_ret != 0 )
           printf( "Preset and Counter Request Error=%d" , val_ret );
      else
           printf( "Quantity=%d Preset=%d", qnta , cont );
      }
}
```

3 Installation

Insert the CD-ROM or the FLOPPY-DISK into the relative drive on the PC, and wait for the autoinstall program to start.
If the autoinstall program does not start, proceed as follows:

- click on the software button `Start` on the applications bar
- select `Settings - Control panel`
- double click on the icon `Install applications` to open the window `Properties - Install applications`
- select the folder `Install/Remove` and press the button `Install`
- follow the instructions provided by the installation program, and press `Next`
- press `End` to run the file `Setup.EXE` on the CD-ROM or FLOPPY
- follow the instructions provided by the installation program, and press the software button `Next` every time you are required to do so
- press the software button `End`

When installation has been completed a box will be displayed asking whether or not you need to install the `Acrobat Reader` software, which allows the package documentation to be displayed in PDF format.

AVVERTENZA: the software that has just been installed will only operate correctly after you have rebooted the PC

3.1 Installing and enabling the XNC connection

Before using the tools in the **CNi PCQUO** software package, make sure that the TCP/IP Communication protocol has been installed on your computer.
If this protocol is not present on your PC, you must use the Windows CDROM (or CDROM update) to install it.

AVVERTENZA: CNi does not supply the Windows CDROM.
To obtain a copy, please contact your normal software salesman.

3.2 Installing the TCP/IP protocol

To install the TCP/IP communication protocol you must:

- open the control panel and click on the icon `Network`
- check that the TCP/IP protocol is not already installed
- press `Add`
- select `Protocol` then click on the button `Add`
- select `Microsoft` in the left hand table and `TCP/IP` in the right hand table

- press the software button Ok